

Task 3 Build a decision tree classifier to predict whether a customer will purchase a product or service based on their demographic and behavioral data. Use a dataset such as the Bank Marketing dataset from the UCI Machine Learning Repository.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
from textblob import TextBlob
data=pd.read_csv('/content/online shopping.csv')
data
```

```
***
```

	Unnamed: 0	CustomerID	Gender	Location	Tenure_Months	Transaction_ID	Transaction_Date	Product_SKU
0	0	17850.0	M	Chicago	12.0	16679.0	1/1/2019	GGOENEBJ079499
1	1	17850.0	M	Chicago	12.0	16680.0	1/1/2019	GGOENEBJ079499
2	2	17850.0	M	Chicago	12.0	16696.0	1/1/2019	GGOENEBQ078999
3	3	17850.0	M	Chicago	12.0	16699.0	1/1/2019	GGOENEBQ079099

```
# Create target variable
data['Purchased'] = data['Online_Spend'].apply(lambda x: 1 if x > 0 else 0)
```

```
features = ['Gender', 'Tenure_Months', 'Quantity',
            'Avg_Price', 'Discount_pct', 'Offline_Spend']
```

```
X = data[features]
y = data['Purchased']
```

```
le = LabelEncoder()
X['Gender'] = le.fit_transform(X['Gender'])
```

```
/tmp/ipython-input-4058603485.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
X['Gender'] = le.fit_transform(X['Gender'])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
dt_model = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=42)
```

```

y_pred = dt_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

```

Accuracy: 1.00

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	15876
accuracy			1.00	15887
macro avg	1.00	1.00	1.00	15887
weighted avg	1.00	1.00	1.00	15887

```

plt.figure(figsize=(15,8))
plot_tree(dt_model,
          feature_names=features,
          class_names=['No Purchase', 'Purchase'],
          filled=True)
plt.show()

```

